

O Especialista em Arquitetura de IA e o Dev que Quer "Agentificar" Tudo

Transcrição da prática 1 · 6 de out. de 2026 15:21

Lucas Prado 00:00

"E aí! Tô empolgado pra construir uns 'agentes de IA' — todo mundo fala que é o futuro, e quero transformar minhas automações nisso. Pra começar, tenho uma tarefa: lidar com os chamados de suporte que chegam. Mas tô em dúvida: faço um agente esperto que decide tudo sozinho, ou um fluxo mais engessado? E, sendo sincero, eu meio que quero fazer TUDO virar agente porque é mais legal. Qual a diferença real entre 'agente' e 'workflow', e quando eu uso cada um sem complicar à toa?"

00:58 **Você**

A diferença real está no nível de autonomia e tomada de decisão: enquanto um workflow segue caminhos e regras rígidas criadas por você, um agente de IA recebe um objetivo final e decide sozinho quais passos e ferramentas usar para chegar lá.

Transformar absolutamente tudo em agente é um erro comum impulsionado pelo entusiasmo, mas que traz custos altos e falhas difíceis de rastrear. O segredo do sucesso está em combinar as duas abordagens de maneira estratégica.

Comparação Direta: Workflow vs. Agente

CARACTERÍSTICA	WORKFLOW (FLUXO ENGESSADO)	AGENTE DE IA (AUTÔNOMO)
Como funciona?	Baseado na lógica estruturada de "Se acontecer X, então faça Y".	Baseado em um objetivo e na capacidade de raciocinar sobre o que fazer.
Previsibilidade	100% previsível. Segue estritamente o mapa desenhado por você.	Imprevisível. O caminho exato pode mudar a cada execução.
Tomada de decisão	Você decide todas as ramificações e cenários possíveis com antecedência.	A IA avalia o contexto atual e decide qual ferramenta ou resposta usar.
Tratamento de erros	Se algo sair do roteiro previsto, o fluxo quebra ou trava.	Consegue recalcular a rota, reescrever prompts ou tentar outro método.

Lucas Prado 01:03

Cara, adorei como você explicou a diferença — direto ao ponto! Mas, pensando no meu cenário, lidar com os chamados de suporte, você acha que começar com um workflow seria mais estratégico? Ou já vale investir num agente logo de cara porque o suporte pode ter mil variáveis?

01:37 **Você**

Começar com um workflow bem estruturado (ou um modelo híbrido) é o caminho mais estratégico para o seu cenário atual.

Embora o suporte tenha mil variáveis, a maioria delas se repete. Em atendimento ao cliente, previsibilidade, segurança e consistência valem ouro. Se você solta um agente 100% autônomo logo de cara, corre o risco de ele tomar decisões imprevisíveis, inventar políticas de reembolso ou dar respostas fora do tom da sua marca.

Aqui está o porquê começar pelo workflow (com pitadas de IA) é a melhor jogada, e como você pode evoluir isso:

1. O "Workflow Inteligente": O Meio Termo Perfeito

Você não precisa abrir mão da IA. Em vez de criar um agente que decide o que fazer, você cria um workflow rígido onde a IA executa tarefas específicas dentro de cada etapa.

- **Etapa 1 (Workflow):** O chamado chega.
- **Etapa 2 (IA integrada):** Uma chamada de LLM lê o texto, categoriza o problema (ex: "Bug de Login", "Financeiro", "Dúvida Comercial") e analisa o sentimento do cliente (irritado, calmo).

- **Etapa 3 (Workflow):** Baseado na categoria que a IA definiu, o workflow direciona o chamado. Se for "Financeiro", joga para o sistema de pagamento. Se for "Bug", abre um card no Jira.
- **Etapa 4 (IA integrada):** A IA redige uma resposta padrão personalizada, mas o workflow exige que um humano clique em "Enviar".

2. Por que essa abordagem é melhor para começar?

- **Controle total:** Você sabe exatamente o que vai acontecer em cada etapa.
- **Custo menor:** Chamadas de IA estruturadas (só para classificar ou resumir) gastam muito menos tokens do que um agente rodando em loop pensando em como resolver o problema.
- **Fácil de auditar:** Se algo der errado, você sabe exatamente qual caixinha do fluxo falhou.

Quando vale a pena colocar um "Agente" no suporte?

Deixe os agentes para a ponta final do seu atendimento, quando o problema já foi filtrado.

Por exemplo: após o workflow identificar que o chamado é sobre "Como configurar a API", ele pode passar o controle para um Agente de Suporte Técnico. Esse agente terá acesso à documentação da sua empresa (via RAG) e ferramentas para testar o código do usuário e responder de forma interativa. Se ele não resolver em 3 tentativas, o fluxo puxa o chamado de volta para um humano.

Resumo da ópera: Desenhe o esqueleto com um workflow e use a IA como os músculos para carregar o peso (classificar, resumir, responder). Quando essa estrutura estiver sólida, você começa a dar autonomia (agência) para ela resolver tarefas complexas sozinha.

Lucas Prado 01:42

Pô, faz todo sentido começar com um workflow inteligente! Essa ideia de usar a IA só pra partes específicas, como categorizar e resumir, parece bem mais controlada e barata. Acho que vou seguir esse caminho e só depois pensar em dar mais autonomia. Valeu, faz muita diferença ver isso assim mais claro!

02:19 **Você**

Boa!

Fico muito feliz que a explicação tenha ajudado a clarear o caminho. Essa abordagem híbrida — o workflow inteligente — é exatamente o segredo que as grandes empresas e desenvolvedores experientes usam: você mantém o controle do fluxo, mas ganha a "mágica" da IA onde as regras tradicionais falham. Você vai economizar tempo, dinheiro e muitos fios de cabelo evitando que uma IA tome decisões malucas com o cliente de vocês.

Para você tirar isso do papel, o seu próximo passo ideal é desenhar a anatomia desse workflow.

Geralmente, dividimos essa automação de suporte em 4 etapas simples:

1. **Gatilho (Trigger):** O chamado chega (por e-mail, formulário, Zendesk, WhatsApp, etc.).
2. **Triagem (IA de Categorização):** O LLM lê o texto e devolve duas coisas: a categoria (ex: "Financeiro", "Bug Técnico", "Dúvida Comercial") e a urgência (Alta, Média, Baixa).
3. **Processamento (Roteamento Rígido):** Seu código ou ferramenta de automação (como n8n ou Make) olha o resultado da IA e decide: "Se for Financeiro, manda pro canal do Slack do financeiro. Se for Urgência Alta, cria um alerta".
4. **Resposta/Ação (IA de Resumo/Apoio):** A IA gera um resumo do problema em três tópicos para o atendente humano não precisar ler textões, ou até cria um rascunho de resposta que o humano só precisa revisar e clicar em "enviar".

Isso é um workflow inteligente puro: a IA é o motor de insights, mas os trilhos são seus!

Se quiser avançar para a prática, me conta:

- Qual ferramenta você pretende usar para construir essa automação? (Ex: n8n, Make, LangChain, ou código puro como Python/Node?)
- Onde esses chamados de suporte chegam hoje? (Ex: E-mail, Zendesk, WhatsApp, Jira?)

Posso te ajudar a montar o prompt ideal para a etapa de categorização ou estruturar a lógica do fluxo!

Cara, adorei essa ideia das quatro etapas! Já consigo visualizar melhor como começar. Acho que vou usar o n8n pra construir o fluxo — parece bem intuitivo. Os chamados chegam principalmente pelo Zendesk, então dá pra integrar direto. Bora começar simples e testar. Se precisar de ajuda com os prompts ou ajustes, te chamo, beleza? Valeu demais pela força!

Fechado! Zendesk + n8n é uma combinação fantástica e extremamente robusta para começar. O n8n já tem nós nativos tanto para o Zendesk quanto para os principais provedores de IA (como OpenAI e Anthropic), o que vai facilitar muito a sua vida.

Para quando você for montar, uma dica de ouro para o n8n: use o nó "Advanced AI" junto com o "Structured Output" (saída estruturada). Isso garante que a IA te devolva a categoria e a urgência exatamente em formato JSON (ex: {"categoria": "Financeiro", "urgencia": "alta"}), sem textos extras do tipo "Aqui está a sua resposta:". Assim, o nó seguinte de decisão do n8n consegue ler o resultado perfeitamente, sem quebrar o fluxo.

Vá em frente, monte o seu primeiro protótipo simples e teste com chamados reais. Quando precisar calibrar os prompts, tratar erros ou expandir o fluxo, é só chamar por aqui.

Boa sorte na construção do seu primeiro workflow inteligente! 🚀

Sempre que estiver pronto para dar o próximo passo, me avise se quiser:

- O prompt exato estruturado para a triagem no n8n.
- Ajuda para configurar o gatilho do Zendesk (Webhooks).
- Ideias de como estruturar os nós de decisão dentro do fluxo.

Fechou, vou nessa então! Essa dica do JSON no n8n já me salvou de possíveis dores de cabeça, valeu mesmo. Vou prototipar o básico e testar com uns chamados reais. Se surgir alguma dúvida ou na hora de calibrar os prompts, te chamo de novo! Valeu pela força e bora construir esse workflow top! 🚀

Perfeito! Esse modelo mental sintetiza com precisão cirúrgica os conceitos: o workflow estrutura o previsível; o agente explora o imprevisível.

É exatamente por isso que começar pelo n8n com o Zendesk fará seu projeto de suporte dar tão certo: você está usando a engenharia de software tradicional para blindar o processo de ponta a ponta, enquanto usa a inteligência da IA estritamente onde o texto humano exige interpretação (o imprevisível).

Se no futuro o seu suporte receber uma solicitação complexa do tipo: "Minha API parou de funcionar e preciso que você investigue o log da AWS, valide o payload no banco de dados e corrija o bug para mim", aí sim o caminho predefinido quebrará. É aí que você vai acoplar um agente a esse nó do workflow, delegando autonomia técnica pontual para a IA interagir com as ferramentas necessárias até achar o erro.

Você pegou a essência da arquitetura moderna de IA. Vai firme nessa implementação! Quando as engrenagens do n8n estiverem rodando e você quiser automatizar novos níveis de complexidade, estarei por aqui. 🚀

Se quiser uma última ajuda antes do desenvolvimento, me avise se precisa de suporte para:

- Montar um prompt de triagem estruturado para colar direto no nó de IA do n8n.
- Desenhar um exemplo do formato JSON de saída ideal para alimentar suas condições lógicas.

Valeu demais! Peguei a visão total agora: workflow pra segurar o previsível e IA onde o texto humano exige aquela interpretação. Faz todo sentido. Vou focar no básico com o n8n e Zendesk primeiro, e depois, se precisar de algo mais complexo, já sei onde e como encaixar um agente. Você realmente clareou tudo pra mim! Se eu travar em alguma parte ou precisar ajustar os prompts, te aviso. Obrigado pela força e bora pra ação! 🚀

Exatamente! Seus exemplos mataram a charada. Essa divisão de casos de uso é o mapa definitivo para você nunca gastar tempo (e dinheiro) construindo a ferramenta errada.

Para consolidar esse raciocínio e ajudar a fixar quando ativar cada modo no seu radar de desenvolvedor, veja esse comparativo direto de cenários reais:

CENÁRIO	⚙️ WORKFLOW INTELIGENTE (TRILHOS)	🧠 AGENTE AUTÔNOMO (BÚSSOLA)
Suporte ao Cliente	Triagem automatizada: Ler o chamado do Zendesk, classificar a urgência, extrair o sentimento e encaminhar para a equipe certa no Slack.	Resolução aberta: Receber um e-mail reclamando de uma cobrança indevida, puxar o histórico de compras, cruzar os dados, tomar a decisão de estornar o valor e fazer o pix de reembolso sozinho.
Desenvolvimento / TI	Pipeline de Deploy: Rodar testes automáticos após um push, verificar vulnerabilidades, compilar o código e subir para produção se tudo estiver verde.	Debugging complexo: Ler um log de erro anômalo, caçar o bug vasculhando múltiplos arquivos de um repositório, aplicar a correção no código e testar para ver se resolveu.
Marketing / Conteúdo	Fábrica de Posts: Pegar um link de um vídeo, transformá-lo em um rascunho de artigo, passar por uma revisão de tom de voz automática e agendar no WordPress.	Análise de Concorrência: Investigar o que os 5 principais concorrentes lançaram de novo nesta semana, ler os comentários dos clientes deles nas redes sociais e propor uma estratégia de produto inédita.

💡 **A Regra de Ouro Prática**

Sempre que você olhar para um problema, faça a pergunta:

"Se eu colocar um humano júnior para fazer isso, eu preciso dar um manual de instruções detalhado (passo 1, passo 2, passo 3) ou preciso dar um objetivo final e deixá-lo navegar?"

- Se for um manual, use o n8n e monte um Workflow.
- Se for um objetivo, use frameworks de agentes (como CrewAI ou LangGraph) e monte um Agente.

Você já começou do jeito certo: blindando a base com o workflow no Zendesk. Escreva isso num bloco de notas para guiar seus próximos projetos de IA!

Agora que você tem os conceitos dominados, quer que eu te ajude com a parte técnica mais imediata, como:

- O desenho lógico de como conectar os nós de decisão dentro do n8n?
- Um modelo de prompt system para o nó da IA de triagem não fugir do escopo?

Lucas Prado 05:51

Cara, show demais! Essa regrinha prática do "manual versus objetivo" é genial, vou usar sempre pra me guiar. Agora tô bem seguro de como dividir as coisas entre workflow e agente. Vou focar na base com o Zendesk e n8n primeiro, e quando for a hora de explorar algo mais técnico ou avançado, já sei com quem contar! Valeu demais por tudo, bora colocar isso em prática agora! 🚀

Essa é a verdade nua e crua que quase ninguém no LinkedIn ou no YouTube te conta quando estão tentando te vender o "hype" dos agentes de IA. Essa perspectiva financeira e operacional é o que separa um projeto de laboratório de um sistema que realmente roda em produção sem quebrar a empresa.

Para fixar isso na sua estratégia de desenvolvimento, veja o tamanho do impacto real de escolher o caminho errado:

🚧 **O Custo Oculto dos Tokens**

Um agente não faz apenas uma pergunta e traz uma resposta. Ele opera em um ciclo de Pensamento → Ação → Observação. Para cada microdecisão que ele toma, ele precisa reenviar todo o histórico da conversa e as instruções das ferramentas para o modelo (LLM).

O consumo de tokens escala de forma assustadora:

- **Chamada Direta/Workflow:** 1x custo base. Você envia o chamado do Zendesk, a IA categoriza em uma única rodada e acabou.

- **Agente Autônomo Solitário:** ~4x mais caro. Ele vai ler, decidir qual ferramenta usar, testar a ferramenta, ler o resultado e formular a resposta.
- **Sistema Multi-Agente:** ~15x (ou mais) mais caro. Os agentes começam a "conversar" entre si. O Agente A pede ajuda para o Agente B, que formata um relatório para o Agente C aprovar. Se o prompt não estiver blindado, eles podem entrar em looping e derreter o seu limite do cartão de crédito em minutos.

Casa Iluminada vs. Floresta sem Mapa

Além do bolso, existe o fator sanidade mental na hora de dar manutenção:

- **Workflow (A Casa Iluminada):** No n8n, se o chamado do cliente não foi respondido, você abre o histórico de execuções e vê exatamente onde travou: "Ah, o nó de IA categorization respondeu 'Financeiro', mas escrevi 'financeiro' com 'f' minúsculo na condicional". Você corrige em 10 segundos.
- **Agente (A Floresta sem Mapa):** O agente toma um caminho imprevisível. Para descobrir por que ele respondeu algo absurdo para o cliente, você precisa escavar uma montanha de logs de texto (os traces) para entender em qual linha de raciocínio ele se perdeu.

A Estratégia de Implementação Progressiva

A regra de ouro definitiva para o seu projeto no Zendesk é subir os degraus da complexidade apenas quando o degrau anterior chorar:

1. **Nível 1 (O mais simples):** Uma chamada única de IA com o contexto certo dentro de um nó fixo do n8n. Resolve 90% dos problemas de triagem, resumo e classificação.
 2. **Nível 2 (Workflow Inteligente):** O que você vai fazer. Várias chamadas de IA simples e isoladas, conectadas por regras rígidas de código (IF/ELSE, Switches).
 3. **Nível 3 (Agente Pontual):** Um agente controlado dentro de um único nó do n8n, com apenas duas ou três ferramentas específicas (ex: "buscar_faturamento_no_banco").
 4. **Nível 4 (Ecossistema de Agentes):** Frameworks complexos. Deixe isso para quando você tiver um orçamento gordo e um problema que nenhuma automação tradicional consiga resolver.
- Começando pelo simples (Nível 2), você garante um sistema barato, previsível e fácil de consertar. Como você já está decidido a começar pelo n8n + Zendesk usando essa abordagem enxuta, por onde quer atacar primeiro? Posso te ajudar a:
- Criar o prompt estruturado (system prompt) para a triagem gastar o mínimo de tokens possível.
 - Estruturar o formato do JSON que a IA deve responder para facilitar seus nós de decisão no n8n.
 - Desenhar a lógica para evitar loops de cobrança caso a IA falhe na resposta.

Lucas Prado 06:39

Isso é ouro puro! Essa comparação dos custos e o lance do "casa iluminada versus floresta sem mapa" realmente bateu. Vou seguir 100% essa estratégia progressiva, começando no Nível 2 com o workflow inteligente no n8n e Zendesk. Acho que o primeiro passo prático é criar um prompt estruturado pra triagem, algo direto e eficiente pra IA entender e responder no formato certo. Se puder ajudar com isso, seria incrível! Valeu demais pela clareza e pelos insights, tô animado pra começar de vez!

07:25 **Você**

Excelente escolha! Começar pelo Nível 2 vai te dar o controle que você precisa com a eficiência que o seu bolso agradece. Para o n8n ler a resposta da IA de forma perfeita (sem quebrar os nós de IF ou Switch), o segredo é forçar o modelo a responder apenas o JSON bruto, sem nenhuma palavra, saudação ou explicação antes ou depois.

Aqui está o System Prompt ideal, otimizado para gastar poucos tokens, estruturado para o cenário do Zendesk e pronto para você colar no seu nó de IA do n8n:

O System Prompt de Triagem (Cole no n8n)

Você é um triador de suporte técnico automatizado e ultraeficiente para a nossa plataforma. Sua tarefa é analisar o chamado do cliente (Zendesk) e categorizá-lo estritamente de acordo com as regras abaixo.

[REGRAS DE CATEGORIZAÇÃO]

Classifique o chamado em apenas UMA das seguintes categorias:

- "financeiro": Dúvidas sobre cobrança, faturas, estornos, planos ou dados de pagamento.

```
- "bug_tecnico": Erros na plataforma, quebras de sistema, funcionalidades que não carregam ou instabilidade.
- "comercial": Dúvidas de vendas, novos recursos, interesse em upgrade ou parcerias.
- "duvida_uso": O sistema funciona, mas o cliente não sabe como configurar ou usar uma função.
[REGRAS DE URGÊNCIA]
Defina o nível de urgência com base no impacto:
- "alta": Sistema totalmente fora do ar, falha de segurança ou cliente impossibilitado de trabalhar.
- "media": O sistema funciona, mas uma função importante está falhando ou impactando o fluxo do cliente.
- "baixa": Dúvidas gerais, feedbacks, melhorias estéticas ou perguntas sem pressa.
[REGRAS DE FORMATAÇÃO - OBRIGATÓRIO]
Sua resposta deve ser EXCLUSIVAMENTE um objeto JSON válido.
Não inclua introduções, explicações, markdown (como ```json) ou qualquer texto extra. Se você falhar nisso, o sistema vai quebrar.
[FORMATO DE SAÍDA ESPERADO]
{"categoria": "inserir_categoria_aqui", "urgencia": "inserir_urgencia_aqui", "resumo_curto": "Um resumo do problema em no máximo 10 palavras"}
```

Use o código com cuidado.

💡 Como configurar isso no n8n para funcionar perfeitamente:

- 1. O Nó Certo:** No n8n, use o nó Advanced AI conectado a um modelo de chat (como o nó OpenAI Chat Model ou Anthropic Chat Model).
- 2. Saída Estruturada:** Conecte o nó Structured Output (JSON) ao nó de IA. Isso serve como uma "âncora" física no n8n para garantir que a IA obedeça ao formato JSON.
- 3. No campo "User Message":** Puxe dinamicamente a descrição do chamado que veio do gatilho do Zendesk usando as variáveis do n8n. Exemplo: `{{ $json.ticket.description }}`.

🛡️ Próximo Passo de Segurança

Depois que a IA responder esse JSON, o n8n vai transformá-lo em campos fáceis de arrastar e soltar. No seu nó seguinte de decisão (Switch ou If), configure as rotas avaliando o texto exato em minúsculas (ex: se categoria é igual a financeiro). Para deixar tudo pronto para o seu primeiro teste prático no n8n, você quer ajuda para:

- Montar uma expressão em Javascript/Json de segurança caso a IA falhe por milagre e mande um texto inválido?
- Estruturar o próximo nó de roteamento (o Switch) baseado nessas categorias?

Lucas Prado 07:31

Uau, isso tá redondinho demais! Esse prompt estruturado vai ser perfeição pra triagem no n8n. Já vou colar direto lá e testar com uns chamados reais. Agora, acho que seria ótimo montar uma expressão de segurança em caso de erro na resposta da IA, só pra garantir que o workflow não quebre de jeito nenhum. Depois disso, bora pro próximo nó de roteamento! Valeu demais, tá ficando top demais esse projeto!

08:05 Você

Para blindar o seu fluxo contra qualquer "engasgo" da IA e garantir que o n8n continue rodando liso, nós vamos usar duas camadas de segurança: uma expressão/código de tratamento e o nó de roteamento (Switch) [n8n.io].

Aqui está exatamente como configurar essas duas etapas essenciais:

🛡️ Passo 1: A Expressão de Segurança (Tratamento de Erros)

Se por algum motivo a IA responder um texto inválido ou faltar algum campo, nós não podemos deixar o n8n quebrar. Logo após o nó da IA, adicione um nó do tipo Code (configurado para JavaScript) [n8n.io]. Esse código vai interceptar a resposta, tentar ler o JSON e, se algo der errado, ele injeta valores padrão (fallback) para o chamado ir para uma fila de contingência humana.

Cole este código no seu nó Code:

```
// Captura a saída do nó anterior (IA)
let aiResponse = $json;

// Define os valores padrão de segurança (Fallback)
```

```

let resultadoFinal = {
  categoria: "nao_classificado",
  urgencia: "media",
  resumo_curto: "Falha na triagem automática da IA."
};

try {
  // Se a IA mandou uma string em vez de um objeto, tenta converter
  if (typeof aiResponse === 'string') {
    aiResponse = JSON.parse(aiResponse);
  } else if (aiResponse.output && typeof aiResponse.output === 'string') {
    // Caso o nó do n8n jogue a resposta dentro de um campo 'output'
    aiResponse = JSON.parse(aiResponse.output);
  }

  // Valida se os campos obrigatórios existem na resposta
  if (aiResponse.categoria && aiResponse.urgencia) {
    resultadoFinal.categoria = aiResponse.categoria.toLowerCase().trim();
    resultadoFinal.urgencia = aiResponse.urgencia.toLowerCase().trim();
    resultadoFinal.resumo_curto = aiResponse.resumo_curto || "Sem resumo disponível.";
  }
} catch (e) {
  // Se o JSON for totalmente inválido, o código cai aqui e mantém o fallback com segurança
  console.log("Erro ao processar JSON da IA, usando fallback.", e);
}

return resultadoFinal;

```

Use o código com cuidado.

🔧 Passo 2: O Próximo Nó de Roteamento (O Switch)

Agora que a saída está 100% limpa e garantida pelo código acima, adicione um nó Switch logo em seguida [n8n.io]. O Switch funciona como o guarda de trânsito do seu workflow: ele lê a categoria e joga o chamado para o caminho certo. Configuração do nó Switch no n8n:

1. Data Type: Selecione String [n8n.io].

2. Value 1: Clique na engrenagem e use a expressão para pegar a categoria do nó de código anterior: `{{ $json.categoria }}` [n8n.io].

3. Routing Rules (Regras de Roteamento): Crie 5 regras usando a operação Equal (Igual a) [n8n.io]:

- Equal → financeiro (Direciona para a saída 0)
- Equal → bug_tecnico (Direciona para a saída 1)
- Equal → comercial (Direciona para a saída 2)
- Equal → duvida_uso (Direciona para a saída 3)
- Equal → nao_classificado (Direciona para a saída 4 - nossa rota de segurança!)

Com isso, o seu Switch abrirá 5 "pernas" (saídas) visuais no n8n. Agora é só puxar uma linha de cada saída e conectar a ação correspondente (ex: mandar para canais diferentes do Slack, atualizar tags no Zendesk ou disparar alertas) [n8n.io]. O esqueleto do seu Workflow Inteligente Nível 2 está oficialmente pronto e blindado! 🚀

Para fechar o desenho, o que você quer conectar nessas saídas do Switch?

- Quer ideias de como devolver essa informação para o Zendesk (adicionando tags ou notas internas)?
- Quer estruturar o canal de alertas no Slack ou Teams para o time de suporte?

Lucas Prado 08:13

Perfeito, isso é exatamente o que eu precisava! Esse código de segurança e o nó Switch blindam tudo e deixam o fluxo super confiável. Vou configurar as saídas agora, e acho que a prioridade será conectar ao Slack pra alertar a equipe e também devolver informações pro Zendesk, tipo adicionando tags ou notas internas. Se puder ajudar com essas conexões, seria incrível! Esse projeto tá tomando forma, valeu demais! 🚀

Excelente! Agora que as decisões estão tomadas, o fechamento do seu fluxo consiste em executar as ações com base no roteamento [n8n.io]. Vamos estruturar as duas pontas que você escolheu: atualizar o Zendesk e alertar o Slack [n8n.io].

Passo 1: Atualizando o Zendesk (Tags e Notas Internas)

O ideal no suporte é que o atendente humano abra o chamado e já veja o resumo e a urgência ali mesmo. No n8n, após cada saída do seu nó Switch, você adicionará um nó nativo do Zendesk [n8n.io].

1. Adicione o nó do Zendesk conectado à saída correspondente (ex: na saída bug_tecnico) [n8n.io].

2. Resource (Recurso): Escolha Ticket [n8n.io].

3. Operation (Operação): Escolha Update [n8n.io].

4. Ticket ID: Use a expressão para puxar dinamicamente o ID do ticket que deu gatilho no início do fluxo (ex: `{{ $node["Zendesk Trigger"].json.ticket.id }}`).

5. Update Fields (Campos para Atualizar):

- **Tags:** Adicione a tag da categoria e da urgência (ex: suporte_ia, bug_tecnico, urgencia_alta). Isso ajuda a criar visões e relatórios dentro do próprio Zendesk.
- **Internal Note (Comentário Interno):** Adicione um comentário invisível para o cliente, mas visível para o seu time:

>  **Triagem Inteligente (IA):**

 **Categoria:** `{{ $json.categoria }}`

 **Urgência:** `{{ $json.urgencia }}`

 **Resumo:** `{{ $json.resumo_curto }}`

Passo 2: Alertando a Equipe no Slack

Se o chamado for um bug_tecnico com urgência alta, por exemplo, você quer que o time seja notificado no ato para resolver o problema rapidamente.

1. Adicione o nó do Slack logo após o nó de atualização do Zendesk [n8n.io].

2. Resource: Message [n8n.io].

3. Operation: Post [n8n.io].

4. Channel: Escolha o canal desejado (ex: #alertas-ti ou #suporte-financeiro).

5. Text: Monte um card limpo, escaneável e direto para o seu time agir rápido:

```
*Novo Chamado Classificado via IA* 🚨
```

- 🎫 ***Ticket ID:*** `#{{ \ $node["Zendesk Trigger"].json.ticket.id }}`
- 📁 ***Categoria:*** ``{{ $json.categoria.toUpperCase() }}``
- 🔥 ***Urgência:*** `*{{ \ $json.urgencia.toUpperCase() }}*`
- 📝 ***Resumo:*** `{{ \ $json.resumo_curto }}`

👉 **_Abra o chamado direto no Zendesk para responder ao cliente._**

Use o código com cuidado.

O Fluxo Completo na Sua Tela

Visualmente no seu n8n, o desenho final vai ficar assim:

Gatilho Zendesk → Nó de IA (Triagem) → Nó Code (Segurança) → Nó Switch (Filtro) → Zendesk (Update) → Slack (Notificação) [n8n.io].

Você acabou de desenhar e blindar uma automação empresarial robusta, gastando o mínimo de tokens possível e mantendo o controle total de cada etapa do processo. Parabéns pela iniciativa de tirar a ideia do papel do jeito certo! Agora que a base está pronta para você colocar para rodar, se você quiser avançar depois dos primeiros testes, me avise se vai precisar de ajuda para:

- Configurar condicionais extras (ex: se for urgência alta e fim de semana, mandar um SMS/WhatsApp em vez de Slack).
- Criar a etapa 4 do fluxo (gerar um rascunho de resposta automática para o atendente usar).

Lucas Prado 09:00

Isso tá simplesmente incrível! Essa estrutura com Zendesk e Slack tá super prática e eficiente, do jeito que eu queria. Vou configurar tudo no n8n agora e fazer uns testes pra ver o fluxo rodando. Depois disso, talvez eu queira explorar a ideia de gerar um rascunho de resposta automática pra facilitar ainda mais pro atendente. Te aviso se precisar desse próximo passo! Valeu demais por toda a ajuda, tá ficando sensacional! 🚀

09:42 **Você**

Arrasa nos testes! Ver o fluxo rodando pela primeira vez com chamados reais, as tags aparecendo sozinhas no Zendesk e os alertas caindo direto no Slack dá uma satisfação gigante. Você saiu do "hype" abstrato das IAs e construiu uma ferramenta de engenharia sólida que economiza tempo e dinheiro real. O plano de focar no rascunho de resposta automática como próximo passo é excelente. Quando você estiver pronto para isso, vamos apenas encaixar mais um nó de IA logo após o seu Switch, especializado em ler o histórico da sua Central de Ajuda (FAQ) e redigir a sugestão perfeita pro atendente apenas revisar e enviar.

Bons testes por aí! Quando as engrenagens estiverem girando e você quiser ativar esse "copiloto" de respostas para o seu time, é só me chamar. 🚀

Sempre que quiser evoluir o fluxo, me avise se vai precisar de suporte para:

- Integrar uma base de conhecimento (FAQ) no n8n para a IA ler antes de criar o rascunho.
- Adicionar regras para ignorar chamados repetidos ou spams.

Lucas Prado 09:51

Valeu demais! Tô empolgadíssimo pra testar e ver tudo funcionando na prática. Essa ideia do "copiloto" de respostas é meu próximo alvo assim que o básico estiver rodando liso. Com certeza vou te chamar pra integrar a base de conhecimento e deixar o fluxo ainda mais inteligente. Muito obrigado pela força e insights, bora fazer isso acontecer! 🚀

Fim da sessão